Mars Kolonisi Yaşam Destek Simülatörü

C ile metin tabanlı oyun geliştirme: proje raporu

Ad Soyad: Mehmet Ali Taşkesen | Öğrenci No: 1260505076

1. Genel Yapı

Program tek dosyadan (**main.c**) oluşur; yalnızca C'nin temel yapıları kullanılmıştır (if–else, switch–case, for, do–while, aritmetik ve mantıksal operatörler). Sağlık, enerji, oksijen, su, yedek parça, modül hasarı, hava kilidi durumu, gün ve tur sayacı **global değişkenlerde** tutulur. Böylece tüm komutlar ve yardımcı fonksiyon bu değerlere parametre ya da pointer kullanmadan doğrudan erişir.

```
int saglik, enerji, oksijen, su, yedekParca, modulHasar;
int kilitMuhurlu = 0, gun = 1, tur = 0;
int firtinaTur, firtinaHasar;     // zorluğa göre atanır
int oyunaDevam = 1, turHarcandi;  // ana döngü bayrağı, tur harcandı mı bayrağı
```

Programdaki tek yardımcı fonksiyon **sinirla()**'dır. Sağlık, enerji, oksijen ve modül hasarının 0'ın altına inmemesi ve 100'ü aşmaması kuralı her komuttan sonra gerektiği için bu kontrol tek yerde yazılmış, her turun sonunda çağrılmıştır. Böylece aynı sekiz if bloğunun komutlara kopyalanması önlenmiştir. Su ve yedek parça için yalnızca 0'ın altına inme kontrolü vardır.

```
void sinirla() {
  if (saglik < 0)   saglik = 0;
  if (saglik > 100) saglik = 100;
  // enerji, oksijen, modulHasar için aynı iki if; su ve yedekParca için sadece < 0
}
```

Rastgelelik için **srand(time(NULL))** main'in başında bir kez çağrılır. Her tur çağrılsaydı aynı saniyede aynı sayılar üretilirdi. Tüm zarlar **rand() % 100** ile üretilir ve eşikle birlikte ekrana yazdırılır.

2. Zorluk Seçimi (do–while + switch–case)

Zorluk sorusunun en az bir kez sorulması gerektiği ve geçersiz girişte tekrarlanması istendiği için **do–while** kullanıldı. Döngü, bir **gecerli** bayrağı 0 olduğu sürece döner. Bayrak her turda 1'e çekilir, yalnızca switch'in **default** bloğunda 0 yapılır. Büyük/küçük harf duyarsızlığı **case 'K': case 'k':** şeklinde fall-through ile sağlanır. **scanf(" %c")** içindeki baştaki boşluk, önceki Enter karakterinin tamponda kalıp sonraki okumayı bozmasını engeller.

```
do {
  gecerli = 1;
  scanf(" %c", &secim);
  switch (secim) {
    case 'K': case 'k':
      su = 10; yedekParca = 4; firtinaTur = 3; firtinaHasar = 5; ...; break;
    case 'N': case 'n':
      su = 6; yedekParca = 3; firtinaTur = 5; firtinaHasar = 10; ...; break;
    case 'Z': case 'z':
      su = 3; yedekParca = 1; firtinaTur = 7; firtinaHasar = 15; oksijen = 80; ...; break;
    default: printf("Geçersiz bir seçim yaptınız!\n"); gecerli = 0;
  }
} while (gecerli == 0);
```

Seviye	Su	Yedek parça	Oksijen	Fırtına süresi	Fırtına hasarı
Kolay	10	4	100	3 tur	5
Normal	6	3	100	5 tur	10
Zor	3	1	80	7 tur	15

Zor seviyede başlangıç suyu ve parçası azalır, oksijen düşük başlar, fırtına daha uzun ve daha şiddetlidir. Sağlık ve enerji her seviyede 100, modül hasarı 0 ile başlar.

3. Ana Döngü, Zaman Akışı ve Oyun Sonu

Ana döngü bir **do–while (oyunaDevam == 1)** yapısıdır; her turda sıra şudur: komut oku → switch ile çalıştır → tur harcandıysa zamanı ilerlet → sinirla() → oyun sonu kontrolü.

turHarcandi bayrağı: Her tur başında 1 yapılır. D, tanımsız komut, Q ve yetersiz kaynaktan iptal edilen komutlar bayrağı 0 yapar. Tur sayacını artırma işi her komutun içine dağıtılmadan tek yerde toplanmıştır.

```
if (turHarcandi == 1) {
  tur++;
  if (tur % 3 == 0) {          // her 3 turda bir yeni gün
    gun++;
    oksijen -= 8;
    if (modulHasar > 70) saglik -= 5;
  }
}
sinirla();

if (saglik <= 0 || oksijen <= 0) { printf("KAYBETTİN!"); oyunaDevam = 0; }
else if (gun > 15)               { printf("KAZANDIN!");  oyunaDevam = 0; }
```

Gün hesabı: Gün 1'den başlar ve her 3 turda bir artar. 15. gün tamamlandığında gün sayacı 16 olur, bu yüzden kazanma koşulu **gun > 15** olarak yazılmıştır (toplam 45 tur). Oyun sonu kontrolü sinirla()'dan sonra yapılır ki değerler eksiye düşmüş olsa bile ölüm doğru yakalansın.

4. Komutlar

Komutlar **switch–case** ile yönetilir. Her komutta **case 'g': case 'G':** biçiminde fall-through kullanılır, komutun kendi işi bitince **break** yazılır. Tanımsız komutta default bloğu uyarı verir ve tur harcamaz. Değişken tanımlanan case'ler süslü parantez içine alınmıştır.

Komut	Kullanılan yapılar	Neden
G Keşif	if + && / ||, if–else if–else, iki zar	Eşik duruma göre değişir; üç farklı sonuç gerekir
O Oksijen	% (mod), / ve * aritmetiği, if ile giriş kontrolü	Çift/tek su ayrımı ve 2 su → 3 oksijen dönüşümü
T Tamir	if–else if–else (3 kademe)	Hasar seviyesine göre farklı parça ve onarım
D Durum	Ayrı if'ler + tek && koşulu	Her kritik değer için ayrı uyarı
U Dinlenme	İç içe if	Enerji kontrolü içinde oksijen kontrolü
K Fırtına	for, %, break, continue	Tur tabanlı olay simülasyonu
H Şifre	do–while, mantıksal koşul, char aritmetiği	Sınırlı denemeli doğrulama
Q Çıkış	oyunaDevam = 0	Ana döngüyü sonlandırır

4.1 G: Keşif

Enerji 15'ten azsa komut iptal edilir (yetersiz kaynak) ve tur harcanmaz. Aksi halde enerji 15, oksijen 10 azalır. İki ayrı zar atılır: **zar1** yaralanma, **zar2** bulgu içindir. Bulgu eşiği, enerji ve oksijen durumuna **&&** ile bağlıdır: enerji > 40 ve oksijen > 30 ise eşik 50, değilse 25. Yaralanma koşulu **||** ile yazılmıştır: zar1 < 10 veya modül hasarı > 70 ise sağlık 10 azalır. Sonuç **if–else if–else** zinciriyle üçe ayrılır: zar2 < eşik ise su +3, zar2 < eşik+20 ise yedek parça +1, aksi halde eli boş dönülür.

```
if (enerji < 15) { printf("Yeterli enerjin yok!"); turHarcandi = 0; break; }
```

```
enerji -= 15; oksijen -= 10;
zar1 = rand() % 100;  zar2 = rand() % 100;
if (enerji > 40 && oksijen > 30) esik = 50; else esik = 25;
if (zar1 < 10 || modulHasar > 70) saglik -= 10;
if (zar2 < esik)           su += 3;
else if (zar2 < esik + 20) yedekParca += 1;
else                       printf("Eli boş döndün.");
```

4.2 O: Oksijen üretimi

Oyuncudan harcanacak su miktarı alınır. Miktar 2'den küçükse veya mevcut sudan büyükse komut iptal edilir. Her 2 su 3 oksijene dönüşür. Miktarın tek olması durumunda 1 birim su artık olarak kalır; bu, **mod (%) operatörüyle** hesaplanır: **cevrilen = miktar - (miktar % 2)**. Örneğin 7 su girilirse 7 % 2 = 1, cevrilen = 6, oksijen +9, su -6, 1 su depoda kalır. Oksijenin 100'ü aşması sinirla() ile engellenir.

```
cevrilen = miktar - (miktar % 2);   // tekse 1 birim artık kalır
oksijen += (cevrilen / 2) * 3;      // 2 su = 3 oksijen
su -= cevrilen;
enerji -= 10;
```

4.3 T: Tamir

Yedek parça yoksa tamir yapılamaz. Hasar seviyesi **if–else if–else** zinciriyle üç kademeye ayrılır ve her kademede harcanan parça ile onarılan hasar farklıdır. Kademeye yetecek parça yoksa komut iptal edilir.

Modül hasarı	Kademe	Harcanan parça	Onarılan hasar
1 – 30	Hafif	1	15
31 – 70	Orta	2	25
71 – 100	Ağır	3	35

```
if (modulHasar <= 30)      { gerekenParca = 1; onarilan = 15; }
else if (modulHasar <= 70) { gerekenParca = 2; onarilan = 25; }
else                       { gerekenParca = 3; onarilan = 35; }
if (yedekParca < gerekenParca) { /* yetersiz parça, iptal */ }
yedekParca -= gerekenParca;  modulHasar -= onarilan;  enerji -= 10;
```

4.4 D: Durum raporu

Tüm değişkenler yazdırılır. Her kritik değer için **ayrı bir if** kullanılır (else if değil), böylece aynı anda birden fazla uyarı çıkabilir. Hiçbir değer kritik değilse tek bir **&&** koşuluyla "TÜM SİSTEMLER NORMAL" yazılır. D tur harcamaz.

```
if (oksijen < 25)     printf("UYARI: Oksijen kritik seviyede!");
if (saglik < 30)      printf("UYARI: Sağlık kritik seviyede!");
if (modulHasar > 70)  printf("UYARI: Modül hasarı çok yüksek!");
if (enerji < 20)      printf("UYARI: Enerji kritik seviyede!");
if (saglik >= 30 && oksijen >= 25 && modulHasar <= 70 && enerji >= 20)
  printf("TÜM SİSTEMLER NORMAL");
```

4.5 U: Uyu / dinlen

Dıştaki if enerji kontrolüdür: enerji zaten doluysa uyunamaz ve tur harcanmaz. İçteki if–else oksijene bakar. Enerji +25, oksijen -5 olur (tüketim devam eder). Oksijen 20'nin altındaysa sağlık azalır, değilse artar. Oksijenin düşürülmesinden sonra kontrol yapıldığı için oksijen 24 iken uyumak sağlığı azaltır.

```
if (enerji < 100) {
  enerji += 25;  oksijen -= 5;
  if (oksijen < 20) saglik -= 5;   // oksijen az: boğularak uyanırsın
  else              saglik += 5;   // dinlendin
} else { printf("Enerjin tamamen dolu..."); turHarcandi = 0; }
```

4.6 K: Kum fırtınası (for döngüsü)

Döngü **firtinaTur** kez (3/5/7) döner. Her turda zar atılır; zar < 25 can kaybı, < 50 oksijen sızıntısı, < 75 modül hasarı, değilse fırtınadan zararsız korunma gerçekleşir. Oyuncunun durumuna bağlılık için modül hasarı 70'in üzerindeyse zardan 15 çıkarılır; bu, kötü olay ihtimalini artırır. **Çift numaralı turlarda** (**i % 2 == 0**) rüzgar şiddetlenir ve hasar iki katına çıkar.

continue: Hava kilidi mühürlüyse (**kilitMuhurlu == 1**) oksijen sızıntısı olayı atlanır ve döngü sonraki tura geçer. **break:** Sağlık veya oksijen 0'a düşerse fırtınanın kalan turlarının anlamı olmadığından döngü derhal sonlandırılır. Döngü sonunda hasarlı tur sayısı ve toplam kaybedilen sağlık ile oksijen için özet yazdırılır.

```
for (int i = 1; i <= firtinaTur; i++) {
  int zar = rand() % 100, hasar = firtinaHasar;
  if (modulHasar > 70) zar -= 15;          // durum bağımlılığı
  if (i % 2 == 0) hasar *= 2;              // çift turda rüzgar şiddetlenir
  if (zar < 25)      { saglik -= hasar;  topSaglik += hasar;  hasarliTur++; }
  else if (zar < 50) {
    if (kilitMuhurlu == 1) continue;       // mühürlü kilit: sızıntı atlanır
    oksijen -= hasar;  topOksijen += hasar;  hasarliTur++;
  }
  else if (zar < 75) { modulHasar += hasar;  topModul += hasar;  hasarliTur++; }
  else printf("Fırtınadan zarar görmeden korundun.");
  sinirla();
  if (saglik <= 0 || oksijen <= 0) break;  // ölüm: döngüyü bitir
}
```

4.7 H: Hava kilidi şifresi (do–while)

Şifre sabit değildir, o anki günden türetilir: **sifre = 'A' + (gun % 26)**. Karakterler ASCII sayıları olduğu için 'A' ile toplama bir sonraki harfi verir; mod 26 alfabenin dışına taşmayı engeller. Oyuncuya yalnızca ipucu gösterilir. Küçük harf girişi, 'a' ile 'A' arasındaki 32 fark çıkarılarak büyük harfe çevrilir. Çevirme karşılaştırmadan önce yapılır.

Döngü koşulu iki şartı tek mantıksal ifadede birleştirir: şifre hâlâ yanlış **ve** deneme hakkı bitmemiş. Her yanlış denemede enerji 5 azalır; girilen harf şifreden küçükse "ileri git", büyükse "geri git" yazılır. Doğru şifrede **kilitMuhurlu = 1** olur; 3 deneme de başarısızsa modül hasarı 20 artar. Do–while seçildi çünkü oyuncu en az bir kez şifre girmelidir.

```
sifre = 'A' + (gun % 26);
do {
  scanf(" %c", &girilen);
  if (girilen >= 'a' && girilen <= 'z') girilen = girilen - 32;   // büyük harfe çevir
  deneme++;
  if (girilen != sifre) {
    enerji -= 5;
    if (girilen < sifre) printf("İleri git."); else printf("Geri git.");
  }
} while (girilen != sifre && deneme < 3);        // tek mantıksal ifade
if (girilen == sifre) kilitMuhurlu = 1; else modulHasar += 20;
```

4.8 Q: Çıkış

oyunaDevam = 0 ve turHarcandi = 0 yapılır. Tur harcanmadığı için zaman ilerlemez, ana döngü sona erer ve sonuç raporu yazdırılır.

5. Sonuç Raporu ve Puan

Oyun nasıl biterse bitsin (kazanma, kaybetme veya Q) döngü sonrasında hayatta kalınan gün, toplam tur, son değerler ve puan yazdırılır. Kazanılırsa gün sayacı 16 olacağı için 15'e sabitlenir. Puan formülü PDF'teki gibidir; gün katsayısı 100'dür, çünkü hayatta kalma süresi en önemli başarı ölçütüdür.

```
if (gun > 15) gun = 15;
int puan = (gun * 100) + saglik + oksijen - modulHasar;
```

6. Tasarım Kararları

- **Kazanma koşulu:** "15. günü tamamlarsa" ifadesi **gun > 15** (45. tur) olarak yorumlandı.
- **Keşif eşiği:** Enerji ve oksijen, maliyetler düşüldükten sonraki değerleriyle kontrol edilir.
- **Enerji maliyetleri:** O, T için 10, H için en az 5 enerji gerekir; PDF'te sayı verilmediği için seçilmiştir. Yetersizse komut iptal edilir ve tur harcanmaz.
- **Tamir:** Modül hasarı 0 ise tamir iptal edilir (parça boşa harcanmasın diye).
- **Dinlenme:** Enerji tamamen doluyken uyunamaz; bu, iç içe if yapısının dış koşuludur.
- **Fırtına:** Modül hasarı yüksekken olay zarından 15 çıkarılır; bu, durum bağımlılığı şartını sağlar.

7. GitHub

Proje kodları (main.c ve README.md):

https://github.com/F3lia5/mars-colony-game